Memory Notes

Simplified DFI 2.1 Simplified edition - DFI Specification v2.1.1 (Denali Software, 17 June 2010)

October 4, 2026

Table of Contents

1 Simplified Dfi 2 1 Index....11

2 Introduction....12

2.1 What DFI is....12

2.2 Why the MC/PHY split exists....13

2.3 Edition lineage....14

2.4 Memory types covered....15

2.5 Optional features....16

3 Organization....17

3.1 Sub-interface map....17

3.2 Clocking....18

3.3 Frequency ratio....19

3.4 Slices, bytes, and data ordering....20

3.5 LPDDR2 command/address carve-out....21

4 Acronyms....22

5 Control Interface....23

5.1 Control signals....24

5.2 LPDDR2 CA mapping on dfi_address....26

5.3 Defaults and idle values....27

5.4 Timing....28

6 Write Data Interface....29

6.1 Write data signals....30

6.2 Timing model....31

7 Read Data Interface..... 32
7.1 Read data signals..... 33
7.2 Timing model..... 34
8 Update Interface..... 35
8.1 Update signals..... 36
8.2 MC-initiated update..... 37
8.3 PHY-initiated update..... 38
8.4 Concurrent requests..... 39
8.5 Update timing parameters..... 40
9 Status Interface..... 41
9.1 Status signals..... 42
9.2 Initialization use..... 44
9.3 Frequency change use..... 45
9.4 Clock disable..... 46
9.5 Parity..... 47
10 Training Interface..... 48
10.1 Training support modes..... 49
10.2 Read leveling signals..... 50
10.3 Write leveling signals..... 52
10.4 Key timing parameters..... 53
11 Low Power Control Interface..... 54
11.1 Low power signals..... 55
11.2 Wakeup encoding..... 56

11.3 Handshake....57
12 Initialization and Training....58
12.1 Reset and initialization....58
12.1.1 Signals held at default until init_complete....58
12.1.2 LPDDR2 special rule....58
12.1.3 init_start handshake....58
12.1.4 End-of-init ctrlupd_req....59
12.1.5 Initialization timing parameters....59
12.2 Training operations....60
12.2.1 Read leveling....60
12.2.2 Write leveling....60
12.2.3 Responsibility modes....60
12.2.4 Training is not required at init....60
13 Command Mapping....62
13.1 DDR2 command encoding....63
13.2 LPDDR2 command encoding....64
14 Write Carriage....65
14.1 Command to enable relationship....66
14.2 Enable to data relationship....67
14.3 Contiguous and interrupted writes....68
14.4 Frequency ratio packing....69
15 Read Carriage....70
15.1 Command to enable relationship....71

15.2 Enable to data relationship....72
15.3 Contiguous reads....73
15.4 Frequency ratio packing....74
16 Wire-Level Walkthroughs....75
16.1 Teaching model....76
16.2 Walkthrough 1: Activate to Read....77
16.3 Walkthrough 2: Activate to Write....78
16.4 What the traces show....79
17 Parameter Families and Ratio Phasing....80
17.1 Parameter families....80
17.1.1 Who defines what....80
17.1.2 What the families constrain....81
17.1.3 Constants, maxima, and fixed values....81
17.2 Ratio and phasing....82
17.2.1 1:1 matched frequency....82
17.2.2 1:2 frequency ratio....82
17.2.3 1:4 frequency ratio....82
17.2.4 Impact on scheduling....82
17.2.5 Ratio selection....83
18 Parameter Tables....84
18.1 Control....85
18.2 Write data....86
18.3 Read data....87

18.4 Update....88
18.5 Status....89
18.6 Training....90
18.7 Low power....92
19 System Behavior....93
19.1 Low power control....94
19.1.1 Entry sequence....94
19.1.2 Wakeup time adjustment....94
19.1.3 Exit sequence....94
19.1.4 Relationship to DRAM power modes....94
19.2 Update mechanism....95
19.2.1 MC-initiated update....95
19.2.2 PHY-initiated update....95
19.2.3 Why the bus must idle....95
19.2.4 Concurrent updates....95
19.3 Frequency change....96
19.3.1 Acknowledged frequency change....96
19.3.2 Frequency ratio reset....96
19.3.3 Pumice does not implement it....96
20 Revision Deltas....97
20.1 What DFI 2.1 added versus DFI 2.0....97
20.2 What later revisions changed....98
21 DFI 2.1 in the Pumice DDR2/LPDDR2 Controller....99

21.1 Sub-interface support....100
21.2 Clock-domain crossing....101
21.3 Command path....102
21.4 Write and read serializers....103
21.5 Known gaps and future work....104

List of Figures

No figures in this document.

List of Tables

Table 1: DFI edition lineage....14
Table 2: DFI 2.1 sub-interface summary....17
Table 3: Acronyms used in this book....22
Table 4: DFI 2.1 control interface signals....24
Table 5: LPDDR2 CA packing into dfi_address....26
Table 6: DFI 2.1 write data interface signals....30
Table 7: Write data timing parameters....31
Table 8: DFI 2.1 read data interface signals....33
Table 9: Read data timing parameters....34
Table 10: DFI 2.1 update interface signals....36
Table 11: Update interface timing parameters....40
Table 12: DFI 2.1 status interface signals....42
Table 13: Training mode encodings....49
Table 14: Read leveling signals....50
Table 15: Write leveling signals....52
Table 16: Training timing parameters....53
Table 17: DFI 2.1 low power control interface signals....55
Table 18: Low power wakeup encoding....56
Table 19: Initialization timing parameters....59
Table 20: Training responsibility modes....60
Table 21: DDR2 command encoding on DFI control signals....63
Table 22: DFI 2.1 LPDDR2 CA mapping....64
Table 23: LPDDR2 rising-edge command opcode....64
Table 24: Timing parameter families and ownership....80
Table 25: Control timing parameter....85
Table 26: Write data timing parameters....86
Table 27: Read data timing parameters....87
Table 28: Update timing parameters....88
Table 29: Status timing parameters....89
Table 30: Training timing parameters....90
Table 31: Low power timing parameters....92
Table 32: DFI 2.1 additions over DFI 2.0....97
Table 33: Brief preview of later DFI revisions....98
Table 34: Pumice DFI 2.1 sub-interface support....100

List of Waveforms

No waveforms in this document.

1 Simplified Dfi 2 1 Index

Generated: 2026-10-20

2 Introduction

2.1 What DFI is

The DDR PHY Interface (DFI) is the wire-level contract between a DDR memory controller (MC) and a DDR physical interface (PHY). Version 2.1.1, published by Denali Software in June 2010, defines the signals, signal relationships, and timing parameters needed to move commands, write data, and read data across that boundary for DDR1, LPDDR1, DDR2, LPDDR2, and DDR3 memory devices.

The DFI deliberately does not specify everything inside the MC or the PHY. It only standardizes the seam: what control information the MC hands off, how the PHY reports data back, how the two sides agree that initialization is done, and how optional features such as frequency ratio, low power, and training are signaled. Anything else, test logic, extra calibration, PHY-specific gearing, is left to the implementation.

2.2 Why the MC/PHY split exists

A modern memory subsystem usually has three planes: the controller (scheduling, bank state, refresh), the PHY (clock generation, pads, delay lines, serializers), and the DRAM devices. The controller wants a stable programming model; the PHY wants freedom to choose pad timing, delay lines, and clocking. DFI sits between them so that a controller team and a PHY team can work against a shared interface instead of sharing every implementation detail.

The DFI clock can come from either side or from the system. The only hard rule is that every DFI signal must be driven from a register clocked by a rising edge of the DFI clock. How the other side receives it, synchronously or asynchronously, is an implementation choice.

2.3 Edition lineage

DFI edition lineage

Version	Year	What changed
DFI 1.0	2007	Initial release; DDR1/LPDDR1/DDR2 focus
DFI 2.0	2007-2008	DDR3 support, read leveling, write leveling
DFI 2.1	2008-2010	LPDDR2 support, low power, frequency ratio, parity, frequency change, update interface, tphy_wrdata

DFI 2.1 is backward compatible with 2.0 at the signaling level, but a 2.1 MC or PHY may offer optional features that a 2.0 partner does not understand. This book focuses on the 2.1 surface and calls out the optional additions explicitly.

2.4 Memory types covered

DFI 2.1 lists support for DDR1, LPDDR1, DDR2, LPDDR2, and DDR3. Not every signal applies to every type. For example, `dfi_reset_n` is only for DDR3, `dfi_odt` is for DDR2 and DDR3, and the LPDDR2 CA bus rides on `dfi_address` while `dfi_bank`, `dfi_ras_n`, `dfi_cas_n`, and `dfi_we_n` are held idle.

2.5 Optional features

Several 2.1 mechanisms are optional and not required for basic DFI compliance:

- Frequency ratio (1:2 or 1:4 MC:PHY clock ratio)
- Low power control handshaking
- Command parity and `dfi_parity_error`
- Frequency change protocol
- PHY-initiated update (`dfi_phyupd_*`)
- Read and write leveling training

A minimal DDR2 controller can ignore most of these and still be DFI compliant. The pumice DDR2/LPDDR2 controller takes exactly that minimal stance: it drives the command, write-data, read-data, and init-status sub-interfaces and leaves update, training, low power, and frequency change unimplemented in its first revision.

Source: DFI Specification v2.1.1 sections 1.0, 2.0

3 Organization

3.1 Sub-interface map

DFI 2.1 partitions the MC/PHY boundary into seven sub-interfaces. Each group has its own signals, direction, and timing parameters.

DFI 2.1 sub-interface summary

Sub-interface	MC -> PHY	PHY -> MC	Purpose
Control	address, bank, RAS/CAS/WE, CKE, CS, ODT, reset	-	DRAM command reflection
Write data	write data, enable, mask	-	Write payload carriage
Read data	read enable	read data, valid, DNV	Read payload return
Update	ctrlupd_req	ctrlupd_ack, phyupd_req/type, phyupd_ack	Idle-window handshakes
Status	data_byte_disable, dram_clk_disable, freq_ratio, init_start, parity_in	init_complete, parity_error	Init and capability signaling
Training	rdlvl_, *wrlvl_* control	rdlvl_, *wrlvl_* response	Read/write leveling
Low power	lp_req, lp_wakeup	lp_ack	Power-down opportunity

The Control, Write data, and Read data sub-interfaces carry ordinary traffic. Update, Status, Training, and Low power are housekeeping groups that let the PHY pause traffic, report init completion, train delays, or enter low power states.

3.2 Clocking

Every DFI signal is driven from a register on the rising edge of the DFI clock. The specification does not fix where that clock comes from, only that both sides agree on it. In a matched-frequency system the DFI clock and the PHY clock are the same. In a frequency-ratio system the PHY runs at a multiple of the MC clock and the DFI clock is the slower MC clock, while individual phases are sampled on the faster PHY clock.

3.3 Frequency ratio

DFI 2.1 supports three ratios: 1:1, 1:2, and 1:4. The optional `dfi_freq_ratio` signal reports the ratio as `2'b00`, `2'b01`, or `2'b10`; `2'b11` is reserved.

When the ratio is not 1:1, the Control interface, Write data interface, and read data enable are replicated into phase-specific copies with a `_pN` suffix. For example, a 1:2 system can drive `dfi_address_p0` and `dfi_address_p1` in a single MC clock, letting the PHY accept two commands per DFI clock. The MC may place a command on any phase; the PHY must accept commands on all phases.

Read data, read data valid, and read data not valid are different. They are replicated into DFI data word copies with a `_wN` suffix instead of `_pN`. A 1:2 ratio has `_w0` and `_w1`; a 1:4 ratio has `_w0` through `_w3`. The suffix for phase 0 or word 0 is optional.

3.4 Slices, bytes, and data ordering

DFI data signals are organized by byte lanes. `dfi_wrdata_mask[0]` masks `dfi_wrdata[7:0]`, `dfi_wrdata_mask[1]` masks `dfi_wrdata[15:8]`, and so on. The same ordering applies to `dfi_rddata_dnv` and `dfi_data_byte_disable`. The `dfi_wrdata_en` and `dfi_rddata_en` widths are ideally one bit per memory data slice, with bit 0 covering the lowest data segment.

3.5 LPDDR2 command/address carve-out

LPDDR2 does not use `dfi_bank`, `dfi_ras_n`, `dfi_cas_n`, or `dfi_we_n`. Its 10-bit Command/Address (CA) bus is mapped onto `dfi_address` as a flat 20-bit word: rising-edge CA bits on `dfi_address[9:0]` and falling-edge CA bits on `dfi_address[19:10]`. The PHY splits that word into the two DDR CA cycles.

Source: DFI Specification v2.1.1 sections 2.0, 3.0, 3.1, 4.7

4 Acronyms

Acronyms used in this book

Term	Meaning
CA	Command/Address bus (used by LPDDR2)
CS	Chip select
CKE	Clock enable
DFI	DDR PHY Interface
DDR	Double data rate SDRAM (generic)
DNV	Data not valid
DRAM	Dynamic random-access memory
MC	Memory controller
MRW	Mode register write (LPDDR2)
MRR	Mode register read (LPDDR2)
ODT	On-die termination
PHY	Physical interface (pads, clocks, delay lines)
RAS/CAS/WE	Row/column/write strobe signals
RD/WR	Read/write commands
REF	Refresh command
PRE	Precharge command
ACT	Activate command

Source: DFI Specification v2.1.1 Glossary (Section 6.0)

5 Control Interface

The control interface carries the command and address information that the MC wants the PHY to reflect to the DRAM devices. The PHY may delay these signals by `tctrl_delay` cycles, but it must preserve their timing relationship and bit ordering at the PHY-DRAM boundary.

In a frequency-ratio system every control signal becomes phase-specific: `dfi_address_pN`, `dfi_bank_pN`, `dfi_cas_n_pN`, and so on. Phase 0 may drop the suffix. The MC may issue commands on any phase; the PHY must accept commands on all phases.

5.1 Control signals

DFI 2.1 control interface signals

Signal	Direction	Width	Default	What it does
dfi_address	MC -> PHY	DFI address width	no default	Address bus for all control commands. For LPDDR2 it carries the 20-bit CA word.
dfi_bank	MC -> PHY	DFI bank width	no default	Bank select for DDR1/LPDDR1 /DDR2/DDR3; idle for LPDDR2.
dfi_cas_n	MC -> PHY	DFI control width	0x1	Column-address strobe. Idle for LPDDR2.
dfi_cke	MC -> PHY	DFI chip select width	0x0 or 0x1	Clock enable to DRAMs; must be driven in all phases for ratio systems.
dfi_cs_n	MC -> PHY	DFI chip select width	0x1	Chip select, active low.
dfi_odt	MC -> PHY	DFI chip select width	0x0	On-die termination control; required for DDR2/DDR3.
dfi_ras_n	MC -> PHY	DFI control width	0x1	Row-address strobe. Idle for LPDDR2.

Signal	Direction	Width	Default	What it does
dfi_reset_n	MC -> PHY	DFI chip select width	0x0	DRAM reset; required only for DDR3.
dfi_we_n	MC -> PHY	DFI control width	0x1	Write enable. Idle for LPDDR2.

5.2 LPDDR2 CA mapping on dfi_address

For LPDDR2 the 10-bit CA bus is sent on both edges of the clock. DFI 2.1 packs the two 10-bit halves into one `dfi_address` word per command cycle.

LPDDR2 CA packing into dfi_address

dfi_address bit	CA bit / edge
9:0	CA[9:0] on the rising edge
19:10	CA[9:0] on the falling edge

So `dfi_address[0]` is CA0 on the rising edge, `dfi_address[10]` is CA0 on the falling edge, and so on up to `dfi_address[19]`. The minimum `dfi_address` width for LPDDR2 is therefore 20 bits. During initialization the LPDDR2 CA bus must be driven with a NOP until `dfi_init_complete` asserts.

5.3 Defaults and idle values

Most control signals default to the DRAM idle state: RAS, CAS, and WE are high, chip select is high, ODT is low, and reset is low. CKE is memory-dependent; most parts expect CKE low at reset, but Mobile DDR parts may expect it high. For LPDDR2 the classic RAS/CAS/WE/bank signals are unused and must remain at their idle values while `dfi_address` carries the CA-encoded command.

5.4 Timing

The only control-interface timing parameter is `tctrl_delay`, defined by the PHY. It specifies how many DFI clock cycles elapse between an assertion or de-assertion on the DFI control signals and the same transition appearing at the PHY-DRAM boundary. If the DFI clock and memory clock are not phase aligned, this value is rounded up to the next integer.

Source: DFI Specification v2.1.1 sections 3.1, 4.2

6 Write Data Interface

The write data interface moves write payloads from the MC to the PHY. It consists of the data bus `dfi_wrdata`, the byte mask `dfi_wrdata_mask`, and the enable `dfi_wrdata_en`. The interface is unidirectional from MC to PHY.

6.1 Write data signals

DFI 2.1 write data interface signals

Signal	Direction	Width	Default	What it does
dfi_wrdata	MC -> PHY	DFI data width	no default	Write data payload. Valid while `dfi_wrdata_en` is asserted.
dfi_wrdata_en	MC -> PHY	DFI data enable width	0x0	Enable for write data and mask; asserted `tphy_wrlat` cycles after the write command and `tphy_wrdata` cycles before data.
dfi_wrdata_mask	MC -> PHY	DFI data width / 8	no default	Per-byte mask; bit N masks `dfi_wrdata[8N+7:8N]`.

In a frequency-ratio system these signals become `dfi_wrdata_pN`, `dfi_wrdata_en_pN`, and `dfi_wrdata_mask_pN`. Phase 0 may drop the suffix. The write data stream is still expected to begin with phase 0 unless the MC chooses to send it unaligned and uses `tphy_wrdelay`.

6.2 Timing model

Three parameters govern write data timing:

Write data timing parameters

Parameter	Meaning
tphy_wrlat	Cycles from write command on control interface to `dfi_wrdata_en` assertion.
tphy_wrdata	Cycles from `dfi_wrdata_en` assertion to first `dfi_wrdata`/`dfi_wrdata_mask` beat.
tphy_wrdelay	Optional PHY-side delay between enable and data in frequency-ratio systems.

`dfi_wrdata_en` is asserted `tphy_wrlat` cycles after the write command. It must remain asserted for the entire contiguous write data burst. The actual data and mask appear `tphy_wrdata` cycles after `dfi_wrdata_en` asserts. For back-to-back writes the enable can stay asserted across command boundaries.

The DFI 1.0/2.0 model effectively fixed `tphy_wrdata` to 1. DFI 2.1 made it a programmable parameter so PHYs that need earlier notification of pending data can request a larger value.

Source: DFI Specification v2.1.1 sections 3.2, 4.3

7 Read Data Interface

The read data interface returns read payloads from the PHY to the MC. It consists of the read enable from the MC (`dfi_rddata_en`) and the returned data, valid flag, and data-not-valid flag from the PHY.

7.1 Read data signals

DFI 2.1 read data interface signals

Signal	Direction	Width	Default	What it does
dfi_rddata_en	MC -> PHY	DFI data enable width	0x0	Tells the PHY to expect read data; asserted `trddata_en` cycles after the read command.
dfi_rddata	PHY -> MC	DFI data width	no default	Returned read data.
dfi_rddata_valid	PHY -> MC	DFI read data valid width	0x0	Asserted with valid read data for each cycle of the burst.
dfi_rddata_dnv	PHY -> MC	DFI data width / 8	0x0	Per-byte data-not-valid flag; required for LPDDR2.

In a frequency-ratio system `dfi_rddata_en` becomes `dfi_rddata_en_pN`, while `dfi_rddata`, `dfi_rddata_valid`, and `dfi_rddata_dnv` become `dfi_rddata_wN`, `dfi_rddata_valid_wN`, and `dfi_rddata_dnv_wN`. A 1:2 ratio has two word copies; a 1:4 ratio has four. The word 0 suffix is optional.

7.2 Timing model

Read data timing parameters

Parameter	Meaning
trddata_en	Cycles from read command on control interface to `dfi_rddata_en` assertion.
tphy_rdlat	Maximum cycles from `dfi_rddata_en` assertion to first `dfi_rddata_valid` assertion.

The MC asserts `dfi_rddata_en` `trddata_en` cycles after the read command and holds it for the duration of contiguous read data it expects. The PHY may return data any time up to `tphy_rdlat` cycles after the enable; it is allowed to return data earlier by asserting `dfi_rddata_valid` sooner. When valid is asserted the entire DFI read data word must be valid.

`dfi_rddata_dnv` is used only for LPDDR2 and has the same timing as `dfi_rddata_valid`. Bit N correlates to the byte group `dfi_rddata[8N+7:8N]`. It is the PHY's way of marking bytes that should be discarded.

Source: DFI Specification v2.1.1 sections 3.3, 4.4

8 Update Interface

The update interface lets either side ask the other to pause normal traffic so that internal PHY or MC settings can be updated. There are two protocols: an MC-initiated update and a PHY-initiated update. Both require the DFI bus to be idle of control, read, and write traffic while the update is in progress.

8.1 Update signals

DFI 2.1 update interface signals

Signal	Direction	Width	Default	What it does
dfi_ctrlupd_req	MC -> PHY	1 bit	0x0	MC-initiated update invitation; asserted for tctrlupd_min.. tctrlupd_max cycles.
dfi_ctrlupd_ack	PHY -> MC	1 bit	0x0	PHY acknowledges the MC-initiated update; may remain low to ignore.
dfi_phyupd_req	PHY -> MC	1 bit	0x0	PHY-initiated update request; must be acknowledged by the MC.
dfi_phyupd_type	PHY -> MC	2 bits	no default	Selects one of four PHY update time types.
dfi_phyupd_ack	MC -> PHY	1 bit	0x0	MC acknowledges the PHY-initiated update.

8.2 MC-initiated update

The MC asserts `dfi_ctrlupd_req` when it knows the DFI will be idle. The PHY is free to acknowledge or ignore the request. If it acknowledges by asserting `dfi_ctrlupd_ack`, the request stays asserted as long as the acknowledge is high, and the bus remains idle for update-related traffic only. If the PHY ignores the request, `dfi_ctrlupd_ack` stays low and the MC may de-assert the request any time after `tctrlupd_min` and before `tctrlupd_max`.

The DFI specification requires the MC to issue update requests and sets a maximum interval `tctrlupd_interval` between requests. The MC must also assert `dfi_ctrlupd_req` at the end of initialization to signal that init is complete.

8.3 PHY-initiated update

The PHY asserts `dfi_phyupd_req` when it needs the bus quieted. The request is accompanied by `dfi_phyupd_type`, which selects one of four timing parameters: `tphyupd_type0` through `tphyupd_type3`. The type value must remain constant while the request is asserted.

The MC must acknowledge by asserting `dfi_phyupd_ack` within `tphyupd_resp` cycles. The PHY keeps `dfi_phyupd_req` asserted until the update completes, then de-asserts it; the MC de-asserts `dfi_phyupd_ack` on the following cycle. The entire acknowledged window is bounded by the selected `tphyupd_typeX`.

8.4 Concurrent requests

If both `dfi_ctrlupd_req` and `dfi_phyupd_req` are asserted at the same time, either side may acknowledge the other's request. The unacknowledged request may then be de-asserted. This is the only situation in which the PHY is permitted to de-assert `dfi_phyupd_req` without first receiving an acknowledge.

8.5 Update timing parameters

Update interface timing parameters

Parameter	Meaning
tctrlupd_interval	Maximum cycles the MC may wait between ctrlupd_req assertions.
tctrlupd_min	Minimum cycles `dfi_ctrlupd_req` must be asserted.
tctrlupd_max	Maximum cycles `dfi_ctrlupd_req` may be asserted.
tphyupd_type0..3	Maximum cycles `dfi_phyupd_req` may remain asserted after ack, per type.
tphyupd_resp	Maximum cycles from `dfi_phyupd_req` to `dfi_phyupd_ack`.

Source: DFI Specification v2.1.1 sections 3.4, 4.5

9 Status Interface

The status interface handles initialization, clock control, frequency ratio, and optional DIMM parity reporting. Several of its signals are optional and only matter if the system implements the related feature.

9.1 Status signals

DFI 2.1 status interface signals

Signal	Direction	Width	Default	What it does
dfi_data_byte_disable	MC -> PHY	DFI data width / 8	no default	Optional per-byte disable; valid only at initialization.
dfi_dram_clk_disable	MC -> PHY	DFI chip select width	0x0	When high, PHY stops clocking the DRAMs.
dfi_freq_ratio	MC -> PHY	2 bits	no default	Optional static ratio: 00=1:1, 01=1:2, 10=1:4, 11=reserved.
dfi_init_complete	PHY -> MC	1 bit	0x0	PHY is ready to respond to DFI traffic; also frequency-change ack.
dfi_init_start	MC -> PHY	1 bit	0x0 or 0x1	MC indicates status signals are valid; also used to request frequency change.
dfi_parity_in	MC -> PHY	1 bit	0x0 or 0x1	Even/odd parity for control interface; replicated as

Signal	Direction	Width	Default	What it does
				dfi_parity_in_pN.
dfi_parity_error	PHY -> MC	1 bit	0x0	DIMM parity error report; correlated to ERR_OUT#.

9.2 Initialization use

At power-up most command and status signals must be held at their default values until `dfi_init_complete` asserts. The exceptions are signals that have no meaningful default, such as `dfi_address` for most systems, or `dfi_data_byte_disable` and `dfi_freq_ratio`, which the MC defines before or during initialization.

`dfi_init_start` tells the PHY that `dfi_data_byte_disable` and/or `dfi_freq_ratio` are valid. A PHY that depends on these settings may wait for `dfi_init_start` before asserting `dfi_init_complete`. A PHY that does not may assert `dfi_init_complete` earlier. In either case initialization is not complete until both `dfi_init_start` and `dfi_init_complete` have been asserted simultaneously for at least one DFI clock cycle.

9.3 Frequency change use

During normal operation, if both sides support the optional frequency change protocol, the MC may re-assert `dfi_init_start` to request a frequency change. The PHY accepts by de-asserting `dfi_init_complete` within `tinit_start` cycles. The MC holds `dfi_init_start` until the change completes; the PHY re-asserts `dfi_init_complete` within `tinit_complete` cycles after `dfi_init_start` drops.

9.4 Clock disable

`dfi_dram_clk_disable` lets the MC stop the clocks to the DRAM devices. The PHY must hold the DRAM clock at a constant value within `tdram_clk_disable` cycles after the signal asserts, and must resume valid clock edges within `tdram_clk_enable` cycles after it de-asserts.

9.5 Parity

`dfi_parity_in` carries even or odd parity over `dfi_address`, `dfi_bank`, `dfi_cas_n`, `dfi_ras_n`, and `dfi_we_n`. In frequency-ratio systems it is replicated as `dfi_parity_in_pN`. `dfi_parity_error` is returned by the PHY within `tphy_paritylat` cycles when a DIMM detects a parity error.

Source: DFI Specification v2.1.1 sections 3.5, 4.1, 4.6, 4.8, 4.9

10 Training Interface

DFI 2.1 provides a training interface for read leveling and write leveling. Read leveling covers both data eye training and gate training. Write leveling is specific to DDR3. The interface is optional overall, but once a PHY declares support for a training operation, the MC must support all three defined modes for that operation.

10.1 Training support modes

Each training capability is announced by a 2-bit mode signal:

Training mode encodings

Encoding	Mode
2’b00	Not supported
2’b01	MC Evaluation: MC enables logic, reads results, adjusts delays
2’b10	PHY Evaluation: MC enables logic, PHY evaluates and sets delays
2’b11	PHY Independent: PHY performs training without MC involvement

The MC must be able to operate with any of these modes; the PHY typically supports only one.

10.2 Read leveling signals

Read leveling signals

Signal	Direction	What it does
dfi_rdlvl_req	PHY -> MC	PHY asks to run data eye training.
dfi_rdlvl_gate_req	PHY -> MC	PHY asks to run gate training.
dfi_rdlvl_mode	PHY -> MC	Announces data eye training support mode.
dfi_rdlvl_gate_mode	PHY -> MC	Announces gate training support mode.
dfi_rdlvl_en	MC -> PHY	Enables data eye training; also acknowledges `dfi_rdlvl_req`.
dfi_rdlvl_gate_en	MC -> PHY	Enables gate training; also acknowledges `dfi_rdlvl_gate_req`.
dfi_rdlvl_cs_n	MC -> PHY	Chip select active during read leveling.
dfi_rdlvl_edge	MC -> PHY	Selects positive or negative DQS edge for training.
dfi_rdlvl_delay_X	MC -> PHY	Per-slice delay value for data eye training.
dfi_rdlvl_gate_delay_X	MC -> PHY	Per-slice delay value for gate training.
dfi_rdlvl_load	MC -> PHY	One-cycle pulse when delay values have been updated.
dfi_rdlvl_resp	PHY -> MC	Training result or completion indication.

For LPDDR2, `dfi_rdlvl_resp` is used only for gate training in MC Evaluation mode, not for data eye training. For DDR3 it carries sampled DQ level or gate value.

10.3 Write leveling signals

Write leveling signals

Signal	Direction	What it does
dfi_wrlvl_req	PHY -> MC	PHY asks to run write leveling.
dfi_wrlvl_mode	PHY -> MC	Announces write leveling support mode.
dfi_wrlvl_en	MC -> PHY	Enables write leveling; also acknowledges `dfi_wrlvl_req`.
dfi_wrlvl_cs_n	MC -> PHY	Chip select active during write leveling.
dfi_wrlvl_delay_X	MC -> PHY	Per-slice write DQS delay value.
dfi_wrlvl_load	MC -> PHY	One-cycle pulse when delay values have been updated.
dfi_wrlvl_strobe	MC -> PHY	Triggers the PHY write leveling strobe.
dfi_wrlvl_resp	PHY -> MC	Completion or sampled DQ result.

Write leveling is a DDR3-specific function. A system that does not support DDR3 write leveling does not need these signals.

10.4 Key timing parameters

Training timing parameters

Parameter	Meaning
trdlvl_resp / twrlvl_resp	Maximum cycles from request to MC enable assertion.
trdlvl_en / twrlvl_en	Minimum cycles from enable to first load or command.
trdlvl_load / twrlvl_load	Minimum cycles from delay update to next load.
trdlvl_dll / twrlvl_dll	Minimum cycles from load to next read or strobe.
trdlvl_resplat / twrlvl_resplat	Maximum cycles from command/strobe to valid response.
trdlvl_max / twrlvl_max	Maximum cycles the MC waits for a PHY Evaluation response.
trdlvl_rr	Minimum command-to-command delay for read leveling reads.
twrlvl_ww	Minimum strobe-to-strobe delay for write leveling.

Source: DFI Specification v2.1.1 sections 3.6, 4.10

11 Low Power Control Interface

The low power control interface lets the MC tell the PHY when the memory subsystem is expected to remain idle, giving the PHY an opportunity to enter a lower power state. The interface is optional for both sides.

11.1 Low power signals

DFI 2.1 low power control interface signals

Signal	Direction	Width	Default	What it does
dfi_lp_req	MC -> PHY	1 bit	0x0	Informs the PHY of a low-power opportunity.
dfi_lp_wakeup	MC -> PHY	4 bits	no default	Encodes how quickly the MC needs normal operation to resume.
dfi_lp_ack	PHY -> MC	1 bit	0x0	PHY acknowledges the low-power opportunity.

11.2 Wakeup encoding

`dfi_lp_wakeup` is valid only while `dfi_lp_req` is asserted. The encoding doubles the wakeup time on each step:

Low power wakeup encoding

dfi_lp_wakeup	tlp_wakeup cycles
4'b0000	16
4'b0001	32
4'b0010	64
4'b0011	128
4'b0100	256
4'b0101	512
4'b0110	1024
4'b0111	2048
4'b1000	4096
4'b1001	8192
4'b1010	16384
4'b1011	32768
4'b1100	65536
4'b1101	131072
4'b1110	262144
4'b1111	unlimited

The value must remain constant until `dfi_lp_ack` asserts. After that the MC may increase the value, allowing the PHY to enter a deeper low power state, but it may not decrease it. The value at the moment `dfi_lp_req` de-asserts sets the `tlp_wakeup` exit budget.

11.3 Handshake

When the MC detects an idle window it asserts `dfi_lp_req` with a chosen `dfi_lp_wakeup`. The PHY may acknowledge within `tlp_resp` cycles or ignore the request. If it acknowledges, both signals remain asserted while the PHY is in the low power state. After `dfi_lp_req` de-asserts, the PHY has `tlp_wakeup` cycles to resume normal operation and de-assert `dfi_lp_ack`.

The DFI clock must remain valid and constant during the entire low power handshake.

Source: DFI Specification v2.1.1 sections 3.7, 4.11

12 Initialization and Training

12.1 Reset and initialization

DFI 2.1 does not dictate a full power-on reset sequence for the MC or the PHY. It only defines when the PHY is ready to accept DFI traffic: when `dfi_init_complete` asserts. Until then, command and status signals must be held at their default values.

12.1.1 Signals held at default until init_complete

The following signals must not change from their default until `dfi_init_complete` asserts:

- `dfi_address` (except the LPDDR2 NOP requirement)
- `dfi_bank`, `dfi_cas_n`, `dfi_ras_n`, `dfi_we_n`
- `dfi_cke`, `dfi_cs_n`, `dfi_odt`, `dfi_reset_n`
- `dfi_wrdata_en`, `dfi_rddata_en`
- `dfi_rddata_dnv`, `dfi_rddata_valid`
- `dfi_ctrlupd_ack`, `dfi_ctrlupd_req`
- `dfi_phyupd_req`, `dfi_phyupd_ack`
- `dfi_dram_clk_disable`
- `dfi_parity_error`, `dfi_parity_in`
- `dfi_rdlvl_req`, `dfi_rdlvl_gate_req`, `dfi_rdlvl_en`, `dfi_rdlvl_gate_en`
- `dfi_rdlvl_load`, `dfi_wrlvl_en`, `dfi_wrlvl_load`, `dfi_wrlvl_req`, `dfi_wrlvl_strobe`
- `dfi_lp_ack`, `dfi_lp_req`

Signals with no meaningful default, such as `dfi_wrdata`, `dfi_wrdata_mask`, `dfi_rddata`, `dfi_phyupd_type`, and the per-slice delay/response signals, do not need a defined value during initialization.

12.1.2 LPDDR2 special rule

For LPDDR2 the `dfi_address` bus must carry a NOP command until `dfi_init_complete` asserts. The `dfi_bank`, `dfi_cas_n`, `dfi_ras_n`, and `dfi_we_n` signals are unused and must remain at constant idle values.

12.1.3 init_start handshake

`dfi_init_start` is optional. When used, it has two purposes:

1. At initialization it tells the PHY that `dfi_data_byte_disable` and/or `dfi_freq_ratio` are valid.

2. During normal operation it requests a frequency change.

If the PHY depends on the disable or ratio settings, it waits for `dfi_init_start` before asserting `dfi_init_complete`. If not, it may assert `dfi_init_complete` earlier. Either way, init is complete only when both `dfi_init_start` and `dfi_init_complete` are high for at least one DFI clock.

12.1.4 End-of-init ctrlupd_req

The MC must assert `dfi_ctrlupd_req` at the end of initialization, after all required training, to indicate that initialization is complete. This is the first update window the PHY sees.

12.1.5 Initialization timing parameters

Initialization timing parameters

Parameter	Meaning
tinit_complete	Maximum cycles from `dfi_init_start` de-assertion to `dfi_init_complete` re-assertion during frequency change.
tinit_start	Maximum cycles for PHY to de-assert `dfi_init_complete` after a frequency change request.

12.2 Training operations

DFI 2.1 supports read leveling and write leveling. Read leveling is used by both DDR3 and LPDDR2 systems; write leveling is DDR3-specific. The interface is optional overall, but if a PHY declares support for a training operation, the MC must support all three defined modes.

12.2.1 Read leveling

Read leveling has two parts:

- Data eye training: centers the read DQS strobe in the DQ data eye.
- Gate training: places the DQS gate at the correct point in the read preamble.

For DDR3 both operations are used. For LPDDR2 the response signal `dfi_rdlvl_resp` is used only for gate training in MC Evaluation mode; data eye training is not supported via that path for LPDDR2.

12.2.2 Write leveling

Write leveling aligns the write DQS to the memory clock. It is only relevant for DDR3. The MC either evaluates the result itself and adjusts `dfi_wrlvl_delay_X`, or it lets the PHY evaluate and set the delay internally.

12.2.3 Responsibility modes

Training responsibility modes

Mode	MC role	PHY role
MC Evaluation	Enables logic, reads response, writes delay values, pulses load	Provides sampled results
PHY Evaluation	Enables logic and waits for completion	Evaluates and sets delays
PHY Independent	None	Performs training autonomously

The MC must be able to operate in all three modes, even though a given PHY only supports one.

12.2.4 Training is not required at init

The DFI specification does not require any training before `dfi_init_complete` asserts. The PHY must guarantee the integrity of the address and control path before declaring init complete, but it may perform training later under its own control or at the MC's request.

Source: DFI Specification v2.1.1 sections 3.5, 3.6, 4.1, 4.10

13 Command Mapping

The DFI control interface is essentially a delayed, possibly widened, copy of the DRAM command interface. How the MC encodes a command depends on the memory type.

13.1 DDR2 command encoding

For DDR2 the classic command truth table applies directly on the DFI control signals.

DDR2 command encoding on DFI control signals

Command	CS_n	RAS_n	CAS_n	WE_n	Address A10	Address other	Bank
NOP	0	1	1	1	X	X	X
ACT	0	0	1	1	row bit	row	bank
RD	0	1	0	1	0	column	bank
RDA	0	1	0	1	1	column	bank
WR	0	1	0	0	0	column	bank
WRA	0	1	0	0	1	column	bank
PRE (one bank)	0	0	1	0	0	X	bank
PREA (all banks)	0	0	1	0	1	X	X
REF	0	0	0	1	X	X	X
DES	1	X	X	X	X	X	X

`dfi_cke` is driven high during normal operation and manipulated separately for self-refresh or power-down entry. `dfi_odt` is driven according to the DRAM ODT programming for reads and writes.

13.2 LPDDR2 command encoding

LPDDR2 has no RAS/CAS/WE pins. Commands and a scrambled address are multiplexed onto a 10-bit CA bus across two clock edges. DFI 2.1 maps the entire CA word onto `dfi_address` as a flat 20-bit value, while `dfi_bank`, `dfi_ras_n`, `dfi_cas_n`, and `dfi_we_n` are held idle.

DFI 2.1 LPDDR2 CA mapping

dfi_address bit	CA bit / edge
9:0	CA[9:0] on rising edge
19:10	CA[9:0] on falling edge

The rising-edge opcode is decoded from CA0r, CA1r, CA2r, CA3r:

LPDDR2 rising-edge command opcode

Command	CA0r	CA1r	CA2r	CA3r
MRW	0	0	0	0
MRR	0	0	0	1
Refresh per-bank	0	0	1	0
Refresh all-bank	0	0	1	1
Activate	0	1	X	X
Write	1	0	0	X
Read	1	0	1	X
Precharge	1	1	0	1
BST	1	1	0	0
NOP/ Deselect	1	1	1	X

Row, column, bank, and mode-register fields are then placed on the remaining CA pins according to the JEDEC LPDDR2 command truth table. The PHY splits the 20-bit `dfi_address` word into the two 10-bit DDR CA cycles.

Source: DFI Specification v2.1.1 sections 3.1, 4.2; JEDEC JESD209-2F Table 60

14 Write Carriage

A write transaction on the DFI has three parts: the write command on the control interface, the write enable assertion, and the data/mask payload. The PHY is responsible for turning that sequence into the correct DRAM timing.

14.1 Command to enable relationship

The MC asserts `dfi_wrdata_en` `tphy_wrlat` cycles after the write command appears on the control interface. The enable must stay asserted for every cycle that contains write data. For contiguous back-to-back writes the enable can remain high across the command boundary.

14.2 Enable to data relationship

The actual data and mask appear `tphy_wrdata` cycles after `dfi_wrdata_en` asserts. In older DFI versions `tphy_wrdata` was effectively fixed at 1; DFI 2.1 made it programmable so the PHY can ask for earlier notification.

`dfi_wrdata_mask` follows the same timing as `dfi_wrdata`. A mask bit of 1 means “do not write this byte.” Bit N covers `dfi_wrdata[8N+7:8N]`. If the data bus is not a multiple of 8, the most significant mask bit covers the most significant partial byte.

14.3 Contiguous and interrupted writes

When two writes are separated by a bubble, the MC de-asserts `dfi_wrdata_en` for at least one cycle. The PHY uses that gap to separate the two bursts. When writes are back-to-back, the enable stays asserted and the data stream is contiguous.

In a frequency-ratio system the write data is split across phases. The MC may keep the data aligned to phase 0 by using `tphy_wrdelay`, or it may send the data unaligned and let the PHY sort it out.

14.4 Frequency ratio packing

In a 1:2 ratio system a single DFI clock contains two phases of write data. The MC can therefore issue one write command per DFI clock and still supply both halves of the burst on `dfi_wrdata_p0` and `dfi_wrdata_p1`. In a 1:4 ratio system four phases fit in one DFI clock.

Source: DFI Specification v2.1.1 sections 3.2, 4.3

15 Read Carriage

A read transaction on the DFI has two parts: the read command on the control interface, and the returned data on the read data interface. The MC tells the PHY when to expect data; the PHY tells the MC when the data is actually valid.

15.1 Command to enable relationship

The MC asserts `dfi_rddata_en` `trddata_en` cycles after the read command. The enable must stay asserted for the duration of contiguous read data expected from the PHY. A single continuous assertion can cover data from multiple read commands.

15.2 Enable to data relationship

The PHY returns data by asserting `dfi_rddata_valid` with the data. The first valid beat may arrive any time up to `tphy_rdlat` cycles after `dfi_rddata_en`, but the PHY is allowed to return it earlier. When `dfi_rddata_valid` is high, the entire DFI read data word is valid.

For LPDDR2 the PHY also asserts `dfi_rddata_dnv` with the same timing. A high bit in `dfi_rddata_dnv` means the corresponding byte in `dfi_rddata` should be discarded.

15.3 Contiguous reads

Back-to-back reads can be covered by a single `dfi_rddata_en` assertion. The MC only de-asserts the enable when it no longer expects contiguous data. The PHY uses `dfi_rddata_valid` to mark the actual cycles that carry valid data.

15.4 Frequency ratio packing

In a 1:2 ratio system the read data is returned on `dfi_rddata_w0` and `dfi_rddata_w1`, one word per half DFI clock. In a 1:4 system there are four word copies `_w0` through `_w3`. The read data enable is still phase-specific: `dfi_rddata_en_p0`, `dfi_rddata_en_p1`, and so on.

Source: DFI Specification v2.1.1 sections 3.3, 4.4

16 Wire-Level Walkthroughs

These walkthroughs use a tiny teaching model so the DFI timing is visible at a glance.

16.1 Teaching model

- Two banks: B0 and B1.
- One rank, one chip select.
- DFI 2.1, 1:2 frequency ratio, so there are two phases per DFI clock: `_p0` and `_p1`.
- Burst length 4, one data word per phase, so BL4 fits in one DFI clock.
- `tphy_wrlat` = 1, `tphy_wrdata` = 1 for writes; `trddata_en` = 2, `tphy_rdlat` = 5 for reads.
- The controller places ACT on phase 0 and the first subsequent RD/WR on the programmed read or write phase.

16.2 Walkthrough 1: Activate to Read

Assume the MC wants to read from an idle bank B0, row R3.

Cycle 0, phase 0: `dfi_cs_n` = 0, `dfi_ras_n` = 0, `dfi_cas_n` = 1, `dfi_we_n` = 1, `dfi_bank` = B0, `dfi_address` = R3. This is the ACT command. Cycle 0, phase 1 is a NOP: `dfi_cs_n` = 1.

After the DRAM's tRCD has passed, the MC issues a read.

Cycle 2, phase 0: NOP. Cycle 2, phase 1: `dfi_cs_n` = 0, `dfi_ras_n` = 1, `dfi_cas_n` = 0, `dfi_we_n` = 1, `dfi_bank` = B0, `dfi_address` = column C1 with A10 = 0 for no auto-precharge. This is the RD command.

Two cycles after the RD command, at cycle 4, the MC asserts `dfi_rddata_en`. It stays asserted for two phases to cover the BL4 burst.

Cycle 4, phase 0: `dfi_rddata_en_p0` = 1. Cycle 4, phase 1: `dfi_rddata_en_p1` = 1.

The PHY returns the first valid data no later than `tphy_rdlat` = 5 cycles after the enable. Suppose it returns early at cycle 5.

Cycle 5, phase 0: `dfi_rddata_w0` valid, `dfi_rddata_valid_w0` = 1. Cycle 5, phase 1: `dfi_rddata_w1` valid, `dfi_rddata_valid_w1` = 1.

The MC captures the two words and delivers them as the BL4 read response.

16.3 Walkthrough 2: Activate to Write

Assume the MC wants to write to bank B1, row R5.

Cycle 0, phase 0: `dfi_cs_n` = 0, `dfi_ras_n` = 0, `dfi_cas_n` = 1, `dfi_we_n` = 1, `dfi_bank` = B1, `dfi_address` = R5. ACT command. Cycle 0, phase 1: NOP.

After tRCD, the MC issues the write.

Cycle 2, phase 0: NOP. Cycle 2, phase 1: `dfi_cs_n` = 0, `dfi_ras_n` = 1, `dfi_cas_n` = 0, `dfi_we_n` = 0, `dfi_bank` = B1, `dfi_address` = column C2 with A10 = 0. WR command.

One cycle after the write command, at cycle 3, the MC asserts `dfi_wrdata_en_p0` and `dfi_wrdata_en_p1`. Because `tphy_wrdata` = 1, the data follows one cycle later at cycle 4.

Cycle 3, phase 0: `dfi_wrdata_en_p0` = 1. Cycle 3, phase 1: `dfi_wrdata_en_p1` = 1.

Cycle 4, phase 0: `dfi_wrdata_p0` = first data word, `dfi_wrdata_mask_p0` = 0. Cycle 4, phase 1: `dfi_wrdata_p1` = second data word, `dfi_wrdata_mask_p1` = 0.

If the MC wanted to mask the lower byte of the first beat, it would set `dfi_wrdata_mask_p0[0]` = 1.

16.4 What the traces show

The control interface carries the command. The write/read data enables predict when data is expected, and the data valid signal bounds the actual return. The PHY is free to delay read data up to `tphy_rdlat`, but the MC must have scheduled `dfi_rddata_en` based on `trddata_en`. For writes, the command leads the enable by `tphy_wrlat`, and the enable leads the data by `tphy_wrdata`.

Source: DFI Specification v2.1.1 sections 3.1, 3.2, 3.3, 4.2, 4.3, 4.4

17 Parameter Families and Ratio Phasing

17.1 Parameter families

DFI 2.1 timing parameters are named `dfi_t*` in public shorthand, but the specification itself uses names like `tctrl_delay`, `tphy_wrlat`, and `trddata_en`. Each parameter is defined by the MC, the PHY, or the system as a whole, and must remain constant while traffic is in flight. They may be changed only when the DFI bus is idle.

17.1.1 Who defines what

Timing parameter families and ownership

Family	Examples	Defined by	Observable by
Control	tctrl_delay	PHY	MC sees command-to-DRAM delay
Write data	tphy_wrlat, tphy_wrdata, tphy_wrdelay	PHY	MC schedules enable and data
Read data	trddata_en, tphy_rdlat	System / PHY	MC schedules enable, PHY bounds valid
Update	tctrlupd_, *tphyupd_*	MC / PHY	Both sides handshake idle windows
Status	tdram_clk_disable, tdram_clk_enable, tinit_complete, tinit_start, tphy_paritylat	PHY / MC	Both sides
Training	trdlvl_, *twrlvl_*	PHY / MC	Both sides during leveling
Low power	tlp_resp, tlp_wakeup	MC	PHY responds within bounds

17.1.2 What the families constrain

Control timing only has `tctrl_delay`: the uniform delay through the PHY to the DRAM pins. Write data timing constrains the gap from command to enable and from enable to data. Read data timing constrains the gap from command to enable and from enable to the latest valid data. Update timing bounds how long each side may keep the bus idle. Status timing bounds clock disable, init handshake, and parity latency. Training timing bounds command-to-response and delay-load delays. Low power timing bounds how quickly the PHY must acknowledge and exit.

17.1.3 Constants, maxima, and fixed values

A parameter may be a fixed constant, a maximum value, or a value derived from other system settings. For example, `tphy_wrlat` is usually a fixed PHY property, while `tphy_rdlat` is often a system-level maximum. The DFI does not mandate absolute numeric ranges; compatibility depends on the MC and PHY supporting overlapping ranges.

17.2 Ratio and phasing

DFI 2.1 supports matched frequency (1:1) and frequency ratio (1:2 and 1:4) systems. The ratio changes how command, write, and read information is packed into a single DFI clock.

17.2.1 1:1 matched frequency

In a 1:1 system the MC clock and the PHY clock are the same. There is one set of control signals, one write data bus, one read data bus, and one read data enable. All timing parameters are expressed in DFI clock cycles, which are also PHY clock cycles.

17.2.2 1:2 frequency ratio

In a 1:2 system the PHY clock runs at twice the DFI clock. Each DFI clock contains two PHY phases. The MC can therefore place commands on both phases:

- Control signals become `dfi_*_p0` and `dfi_*_p1`.
- Write data and mask become `dfi_wrdata_p0` / `_p1` and `dfi_wrdata_mask_p0` / `_p1`.
- Write data enable becomes `dfi_wrdata_en_p0` / `_p1`.
- Read data enable becomes `dfi_rddata_en_p0` / `_p1`.
- Read data, valid, and DNV become `dfi_rddata_w0` / `_w1`, `dfi_rddata_valid_w0` / `_w1`, and `dfi_rddata_dnv_w0` / `_w1`.

The MC may issue a command on any phase. The PHY must accept commands on all phases.

17.2.3 1:4 frequency ratio

In a 1:4 system there are four PHY phases per DFI clock. The same suffixing applies with `_p0` through `_p3` and `_w0` through `_w3`. This allows the MC to issue up to four commands per DFI clock and to receive up to four read data words per DFI clock.

17.2.4 Impact on scheduling

A higher ratio lets the MC run slower than the DRAM pins while still supplying full pin bandwidth. The trade-off is scheduling complexity: the controller must place each command on the correct phase, align write data to `tphy_wrlat` and `tphy_wrdata` in PHY-clock terms, and align the read data enable to `trddata_en` in PHY-clock terms. The `tphy_wrdelay` parameter exists so the MC can keep write data aligned to phase 0 in its own view while the PHY shifts it to the correct phase internally.

17.2.5 Ratio selection

The ratio is static for normal operation. It is communicated at initialization via `dfi_freq_ratio` and confirmed by the `dfi_init_start` handshake. A later frequency change can select a new ratio, but that requires the optional frequency change protocol.

Source: DFI Specification v2.1.1 sections 2.0, 3.1, 3.2, 3.3, 4.7, 5.0

18 Parameter Tables

The table below lists every timing parameter named in the DFI 2.1 inventory, grouped by sub-interface, with a plain-English meaning.

18.1 Control

Control timing parameter

Parameter	Meaning
tctrl_delay	Cycles from DFI control transition to the same transition at the PHY-DRAM boundary.

18.2 Write data

Write data timing parameters

Parameter	Meaning
tphy_wrdata	Cycles from `dfi_wrdata_en` assertion to first `dfi_wrdata` beat.
tphy_wrdelay	Optional PHY delay between enable and data in ratio systems.
tphy_wrlat	Cycles from write command to `dfi_wrdata_en` assertion.

18.3 Read data

Read data timing parameters

Parameter	Meaning
tphy_rdlat	Maximum cycles from `dfi_rddata_en` to first `dfi_rddata_valid`.
trddata_en	Cycles from read command to `dfi_rddata_en` assertion.

18.4 Update

Update timing parameters

Parameter	Meaning
tctrlupd_interval	Maximum cycles the MC may wait between `dfi_ctrlupd_req` assertions.
tctrlupd_min	Minimum cycles `dfi_ctrlupd_req` must be asserted.
tctrlupd_max	Maximum cycles `dfi_ctrlupd_req` may be asserted.
tphyupd_type0	Maximum cycles `dfi_phyupd_req` may stay asserted after ack for type 0.
tphyupd_type1	Same, for type 1.
tphyupd_type2	Same, for type 2.
tphyupd_type3	Same, for type 3.
tphyupd_resp	Maximum cycles from `dfi_phyupd_req` to `dfi_phyupd_ack`.

18.5 Status

Status timing parameters

Parameter	Meaning
tdram_clk_disable	Cycles from `dfi_dram_clk_disable` assertion to DRAM clocks held constant.
tdram_clk_enable	Cycles from `dfi_dram_clk_disable` de-assertion to first valid DRAM clock edge.
tinit_complete	Maximum cycles from `dfi_init_start` de-assertion to `dfi_init_complete` re-assertion.
tinit_start	Maximum cycles for PHY to de-assert `dfi_init_complete` after a frequency change request.
tphy_paritylat	Maximum cycles from `dfi_parity_in` to associated `dfi_parity_error`.

18.6 Training

Training timing parameters

Parameter	Meaning
trdlvl_dll	Minimum cycles from read-leveling delay load to next read/MRR command.
trdlvl_en	Minimum cycles from `dfi_rdlvl_en` to first load or command.
trdlvl_load	Minimum cycles from delay update to next `dfi_rdlvl_load`.
trdlvl_max	Maximum cycles MC waits for a PHY Evaluation read-leveling response.
trdlvl_resp	Maximum cycles from `dfi_rdlvl_req` to `dfi_rdlvl_en`.
trdlvl_resplat	Maximum cycles from read/MRR command to valid `dfi_rdlvl_resp`.
trdlvl_rr	Minimum command-to-command delay during read leveling.
twrlvl_dll	Minimum cycles from write-leveling delay load to next `dfi_wrlvl_strobe`.
twrlvl_en	Minimum cycles from `dfi_wrlvl_en` to first load or strobe.
twrlvl_load	Minimum cycles from delay update to next `dfi_wrlvl_load`.
twrlvl_max	Maximum cycles MC waits for a PHY Evaluation write-leveling response.
twrlvl_resp	Maximum cycles from `dfi_wrlvl_req` to `dfi_wrlvl_en`.
twrlvl_resplat	Maximum cycles from `dfi_wrlvl_strobe` to valid `dfi_wrlvl_resp`.
twrlvl_ww	Minimum strobe-to-strobe delay

Parameter	Meaning
	during write leveling.

18.7 Low power

Low power timing parameters

Parameter	Meaning
tlp_resp	Maximum cycles from `dfi_lp_req` to `dfi_lp_ack`.
tlp_wakeup	Maximum cycles `dfi_lp_ack` may remain asserted after `dfi_lp_req` de-asserts.

Source: DFI Specification v2.1.1 sections 3.1 through 3.7

19 System Behavior

This chapter covers three DFI 2.1 housekeeping protocols: low power control, the update mechanism, and the optional frequency change handshake.

19.1 Low power control

The DFI 2.1 low power interface is an opportunity handshake, not a forced state transition. The MC tells the PHY that the memory subsystem is idle and how quickly it may need to resume. The PHY decides whether to enter a lower power state.

19.1.1 Entry sequence

1. The MC detects an idle window and asserts `dfi_lp_req`.
2. The MC drives `dfi_lp_wakeup` with the required exit time.
3. The PHY either acknowledges within `tlp_resp` cycles or ignores the request.
4. If acknowledged, both `dfi_lp_req` and `dfi_lp_ack` remain asserted while the PHY is in the low power state.

19.1.2 Wakeup time adjustment

After the PHY acknowledges, the MC may increase `dfi_lp_wakeup`, allowing the PHY to pick a deeper state. It may not decrease the value. The final value at the moment `dfi_lp_req` de-asserts sets the `tlp_wakeup` exit budget.

19.1.3 Exit sequence

1. The MC de-asserts `dfi_lp_req` when it needs normal operation to resume.
2. The PHY has `tlp_wakeup` cycles to exit the low power state and de-assert `dfi_lp_ack`.
3. Normal traffic may resume after `dfi_lp_ack` is low.

19.1.4 Relationship to DRAM power modes

DFI low power is about the PHY, not the DRAM. The DRAM may already be in self-refresh, power-down, or a deep power-down state through separate CKE and command sequencing. The DFI low power interface simply lets the MC and PHY coordinate so that the PHY does not try to issue traffic while the MC thinks it is allowed to sleep. The DFI clock must remain valid and at a constant frequency during the entire handshake.

19.2 Update mechanism

The update mechanism lets the PHY or MC pause ordinary traffic so that internal settings can be refreshed. Typical reasons include temperature compensation of delay lines or recomputing ODT settings. Both protocols require the DFI to be idle of control, read, and write traffic during the update window.

19.2.1 MC-initiated update

The MC asserts `dfi_ctrlupd_req` for at least `tctrlupd_min` and at most `tctrlupd_max` cycles. The PHY may acknowledge with `dfi_ctrlupd_ack`, in which case the request stays asserted as long as the acknowledge is high, or the PHY may ignore it. The MC is required to offer these update windows periodically, bounded by `tctrlupd_interval`.

At the end of initialization the MC must assert `dfi_ctrlupd_req` to mark the end of the init sequence. This is often the first update window the PHY sees.

19.2.2 PHY-initiated update

The PHY asserts `dfi_phyupd_req` with a stable `dfi_phyupd_type` value. The MC must acknowledge with `dfi_phyupd_ack` within `tphyupd_resp` cycles. The PHY keeps the request asserted until the update completes, then de-asserts it. The MC de-asserts `dfi_phyupd_ack` on the cycle after the request drops.

The selected `dfi_phyupd_type` determines which `tphyupd_typeX` bound applies. Type 0 is the shortest; type 3 is the longest.

19.2.3 Why the bus must idle

Updates often change delay lines or termination settings that affect how traffic is driven or sampled. If a read or write were in flight during the update, its timing assumptions could change mid-burst. Both protocols therefore require the MC to place the system in a state where no control, read, or write traffic is moving across the DFI.

19.2.4 Concurrent updates

If both request signals are asserted simultaneously, either side may acknowledge the other's request. The unacknowledged request may then be dropped. This is the only case where the PHY may de-assert `dfi_phyupd_req` without an acknowledge.

19.3 Frequency change

DFI 2.1 includes an optional frequency change protocol. It reuses the initialization handshake signals `dfi_init_start` and `dfi_init_complete`. The protocol exists in the specification, but it is not required for DFI compliance.

19.3.1 Acknowledged frequency change

1. During normal operation, with `dfi_init_complete` high, the MC asserts `dfi_init_start` to request a frequency change.
2. The PHY accepts by de-asserting `dfi_init_complete` within `tinit_start` cycles.
3. The MC continues to hold `dfi_init_start` while the system changes clocks.
4. The PHY re-initializes on the new frequency and re-asserts `dfi_init_complete` within `tinit_complete` cycles after `dfi_init_start` de-asserts.

If the PHY does not de-assert `dfi_init_complete` within `tinit_start` cycles, the MC must abort the request and release `dfi_init_start`.

19.3.2 Frequency ratio reset

When `dfi_init_start` is asserted for a frequency change, both sides must reset their DFI read data word pointers to 0. This ensures that the word suffix alignment is consistent after the clock change.

19.3.3 Pumice does not implement it

The pumice DDR2/LPDDR2 controller does not implement the frequency change protocol. Its `dfi_init_start` is used only during initialization, and the DFI clock ratio is fixed by the PHY integration. A controller that needs runtime frequency scaling would need to add the acknowledged-frequency-change state machine to the DFI plane.

Source: DFI Specification v2.1.1 sections 3.4, 3.5, 3.7, 4.5, 4.8, 4.11; `docs/pumice_mas/ch03_interfaces/02_dfi_v21_interface_spec.md`

20 Revision Deltas

20.1 What DFI 2.1 added versus DFI 2.0

DFI 2.1 is backward compatible with 2.0, but it added several optional features. The most important additions are:

DFI 2.1 additions over DFI 2.0

Feature	What changed
LPDDR2 support	CA bus mapped onto `dfi_address`, `dfi_rddata_dnv` added, read leveling extended for LPDDR2 gate training.
Low power control interface	New `dfi_lp_req`, `dfi_lp_wakeup`, `dfi_lp_ack`, plus `tlp_resp` and `tlp_wakeup`.
Frequency ratio	Formalized 1:2 and 1:4 ratios with `_pN` phase copies and `_wN` read data copies.
Parity interface	`dfi_parity_in` and `dfi_parity_error` for DDR3 DIMM command parity.
Frequency change protocol	Optional runtime frequency change via `dfi_init_start` / `dfi_init_complete`.
Update interface	Formalized `dfi_ctrlupd_*` and `dfi_phyupd_*` idle-window handshakes.
tphy_wrdata	Made the enable-to-data delay programmable instead of fixed at 1.
dfi_data_byte_disable	Optional per-byte disable at initialization.

None of these are required for DFI compliance. A 2.1 MC and a 2.0 PHY can still interoperate if they only use the common subset.

20.2 What later revisions changed

DFI 3.1 and subsequent revisions moved the standard toward higher data rates and more complex memories. The brief preview below is based on the public direction of later DFI versions; this book is scoped to DFI 2.1.

Brief preview of later DFI revisions

Area	Later change
Error reporting	Added `dfi_error` and `dfi_alert_n` style reporting, plus data CRC handling for DDR4/LPDDR4.
Multiple channels	Introduced channel indexing for multi-channel PHYs and byte groups.
Richer training	Added DBI training, PHY master mode, and more per-bit/per-byte training controls.
CA parity	Extended command parity to LPDDR4 CA buses.
Frequency change	Refined the frequency change handshake and added gear ratio programming.

For the pumice DDR2/LPDDR2 controller these later additions are out of scope. The controller targets only the DFI 2.1 command, write-data, read-data, and init-status sub-interfaces.

Source: DFI Specification v2.1.1 sections 1.0, 3.4, 3.5, 3.6, 3.7, 4.7, 4.8, 4.9, 4.10, 4.11

21 DFI 2.1 in the Pumice DDR2/LPDDR2 Controller

This book is a simplified study guide for the DFI 2.1 interface used by the pumice DDR2/LPDDR2 memory controller. The controller's DFI plane is implemented in `rtl/macro/pumice_dfi_plane.sv` and documented in the project's microarchitecture and interface specifications.

21.1 Sub-interface support

Pumice DFI 2.1 sub-interface support

Sub-interface	Used by pumice?	Notes
Control	Yes	Per-phase command placement for DDR2 RAS/CAS/WE and LPDDR2 CA.
Write data	Yes	Serialized write data, mask, and enable at `t_phy_wrlat`.
Read data	Yes	Read data enable at `t_rddata_en`; capture on `dfi_rddata_valid`.
Status init	Yes	`dfi_init_start` / `dfi_init_complete` only.
Update	No	`ctrlupd_*` / `phyupd_*` not driven in the first revision.
Training	No	PHY self-trains at startup (`a7ddrphy`).
Frequency change	No	No use case in the current design.
Low power	No	No PHY-side low-power coordination.
Parity / error	No	Not needed for DDR2/LPDDR2 here.

21.2 Clock-domain crossing

Unlike the classic model where DFI sits on the controller clock, pumice places the DFI datapath on a dedicated `dfi_clk` and puts the single controller-to-PHY crossing inside `pumice_dfi_plane`. The crossing uses `gaxi_fifo_async` instances for command, write data, read data, and init event tokens. There are no standalone synchronizers.

21.3 Command path

`pumice_dfi_cmd_path` pops a command FIFO and places the active JEDEC command on the programmed write or read phase, filling the other phases with NOP. For DDR2 it uses the classic RAS/CAS/WE encoding. For LPDDR2 it uses the CA-bus formatter described in `docs/uarch/LPDDR2_CA_ENCODING.md`, packing rising CA on `dfi_address[9:0]` and falling CA on `dfi_address[19:10]` while holding the RAS/CAS/WE/bank signals idle.

21.4 Write and read serializers

`pumice_dfi_wr_serializer` presents `dfi_wrdata`, `dfi_wrdata_en`, and `dfi_wrdata_mask` at the programmed write latency. `pumice_dfi_rd_aligner` drives `dfi_rddata_en` at the programmed read enable latency and captures `dfi_rddata` when `dfi_rddata_valid` asserts, packing whole words into the async FIFO back to the controller clock domain.

21.5 Known gaps and future work

The first pumice DFI plane intentionally omits update handshaking, training control, low-power coordination, and frequency change. These could be added in a later revision without changing the command/write/read datapath. A future DDR3/LPDDR3 controller would need a larger DFI sub-interface set and would likely replace the DFI plane rather than extend this one.

Source: `docs/uarch/PUMICE_DFI_PLANE_UARCH.md`, `docs/pumice_mas/ch03_interfaces/02_dfi_v21_interface_spec.md`, `docs/uarch/LPDDR2_CA_ENCODING.md`